Design and Implementation of a Distributed Confidential Query Protocol for Spark

A Thesis Submitted

to School of Computer Science and Technology of

Northwestern Polytechnical University

in Partial Fulfillment of the Requirements

for the Degree of Bachelor of Engineering

by

Md Rakibul Islam Raihan

Advisor: Chen Yaxing

Abstract

In this paper, we introduce the Distributed Confidential Query Protocol (DCQP) developed specifically for Apache Spark, putting forth an improved way of enhancing data security in distributed computing environments. This is done by using Trusted Execution Environments (TEEs) to ensure data confidentiality during complex processing pipelines of Spark. DCQP provides a strong defense against unauthorized access and possible breaches through robust encryption techniques, strict access controls and advanced authentication mechanisms. More so, it can be easily integrated into the cluster architecture of Spark allowing seamless execution of private inquiries over huge distributed datasets. This article gives an in-depth analysis on the principles behind the design and practice of implementing DCQP which explains how it plays a crucial role in making Spark's data safe. Additionally, a set of extensive experimental evaluations has been carried out that fully indicate how well DCQP works towards ensuring protection for sensitive information as well as maintenance of optimum performance and scalability in cloud-based deployments.

KEY WORDS: query, spark, confidential, implement, cluster

摘要

在本文中，我们介绍了专门为 Apache Spark 开发的分布式机密查询协议（DCQP），提出了一种改进的方法来增强分布式计算环境中的数据安全性。这是通过使用可信执行环境（TEE）来确保 Spark 复杂处理管道中的数据机密性来实现的。DCQP 通过强大的加密技术、严格的访问控制和先进的身份验证机制，对未经授权的访问和可能的违规行为提供了强有力的防御。更重要的是，它可以很容易地集成到 Spark 的集群架构中，允许在巨大的分布式数据集上无缝执行私有查询。本文深入分析了实现 DCQP 的设计和实践背后的原则，解释了它如何在确保 Spark 数据安全方面发挥关键作用。此外，还进行了一系列广泛的实验评估，充分表明 DCQP 在确保敏感信息的保护以及在基于云的部署中保持最佳性能和可扩展性方面的工作效果。

关键词： query, spark, confidential, implement, cluster

Table of Contents

Title Page ...……...1

Abstract ……2

Table of Contents ……...…...4

Chapter One Introduction ….…….....6

1.1 Introduction of Distributed Confidential Query Protocol for Spark ….....…...6

1.1.1 Background of Distributed Confidential Query Protocol for Spark ……6

1.1.2 Motivation of Distributed Confidential Query Protocol for Spark ….....6

1.1.3 Encryption Techniques ……..……...……...….....8

1.1.4 Distributed Query Processing …...…...……...….....8

1.2 Core Components ……...….....8

1.2.1 Query Executor ……..……...8

1.2.2 Encryption Levels …….…….....8

1.2.3 Key Management ....…...……...…...…9

1.2.4 Secure Communications ……9

1.3 Use Cases ……....9

Chapter Two Preliminaries ……...11

2.1 Distribute System ……...…….…11

2.2 Confidential Query Protocol …….....11

2.3 Apache Spark ……...……...……...…….12

2.4 Design and Implementation of Spark Architecture ……...14

Chapter Three Architecture Design ……17

3.1 Problem Description and Design Philosophy …...……17

3.1.1 Problem Description and Overall Design Concept ...……...18

3.1.2 Design Philosophy of The System Architecture ……...……20

3.1.3 Deployment Diagram of Architecture Design **……21**

3.2 Systematic Structure ……...…21

3.3 Logical Plan ……23

3.4 Physical Plan ……23

3.5 Distribution ……...23

3.6 Reception ……...…...23

3.7 Task Execution ……...…...24

3.8 Shuffle ……...27

Chapter Four Prototype Implementation ……31

4.1 Overall Conceptual Scheme ……31

4.2 Detailed Logical Scheme ……..……33

4.3 Executing the Physical Plan ……36

Chapter Five Experiment ……...…...39

5.1 Initial Setup ……..….39

5.2Specific Experimental Details ……..….40

5.3 Key Findings ……..…43

Chapter Six Conclusion …….....45

6.1 Limitations of the Current Work (DCQP) ……...…...45

6.2 Future Work Directions ……...46

6.3 Summary of Key Findings ……...47

References ……...……...…….49

Acknowledgement ……...…...52

Chapter One Introduction

1.1 Introduction of Distributed Confidential Query Protocol for Spark

1.1.1 Background of Distributed Confidential Query Protocol for Spark

Distributed Confidential Query Protocol (DCQP) for Apache Spark is a framework designed to offer stable, privacy-preserving distributed query processing on a Spark cluster. Its aim is to solve the problem of triggering questions when processing sensitive information and to ensure that the facts are preserved. exclusive and personal throughout the processing pipeline. It adopts Intel SGX to establish hardware-assisted enclaves in the untrusted cloud so as to protect the confidentiality and integrity of sensitive data run inside[1]. With the increasing adoption of large-scale information analytics, there is usually a need for companies to investigate sensitive or private facts that include personal data or proprietary commercial business information. DCQP resolves this assignment using query processing mechanisms over securely encrypted distributed statistics. In today's data-driven world, the ability to analyze large datasets efficiently while ensuring data privacy and security is of paramount importance. Apache Spark has emerged as a powerful framework for distributed data processing, offering speed, scalability, and ease of use. However, as data processing tasks become increasingly complex and sensitive, the need for robust confidentiality mechanisms becomes critical. The aim of this research is to address the challenge of performing confidential queries on distributed data within the Apache Spark framework. Specifically, we focus on designing and implementing a novel distributed confidential query protocol tailored for Spark environments. This protocol aims to enable secure and efficient execution of queries while preserving the confidentiality of sensitive data. By developing a distributed confidential query protocol for Spark, we seek to provide a practical solution for organizations and researchers dealing with sensitive data. This protocol will facilitate secure data analysis and processing in distributed environments, enabling users to derive valuable insights from their data without compromising

privacy or security. Through this research, we aim to contribute to the growing body of knowledge in the field of distributed systems and data privacy. Our findings have the potential to impact various industries and domains, including healthcare, finance, and cybersecurity, where the confidentiality of data is of utmost importance. Overall, our research endeavors to bridge the gap between the need for efficient data processing and the imperative of data privacy, thereby advancing the state-of-the-art in distributed computing and data security.

1.1.2 Motivation of Distributed Confidential Query Protocol for Spark

The Distributed Confidential Query Protocol (DCQP) for Spark emerges from the increasing necessity to handle vast amounts of sensitive data securely in distributed computing environments. As data analytics and machine learning applications proliferate, the demand for systems that can process large datasets while maintaining confidentiality grows. Spark, a leading distributed computing framework, provides robust data processing capabilities but lacks built-in mechanisms for ensuring data privacy in a distributed context. DCQP addresses this gap by integrating confidentiality measures directly into Spark's processing paradigm. The primary motivation for DCQP is to enable secure multi-party computations where data from different sources can be analyzed collectively without exposing sensitive information. This is particularly critical in industries such as healthcare, finance, and government, where data privacy regulations like GDPR and HIPAA mandate stringent protection of personal and sensitive information. DCQP ensures that data remains encrypted during processing, thus preventing unauthorized access and leakage of confidential information. Furthermore, DCQP leverages advanced cryptographic techniques to support secure queries across distributed datasets. By doing so, it allows organizations to harness the full power of distributed computing without compromising on data security. This protocol enables collaborative analytics, where multiple parties can contribute to and benefit from joint data analysis without revealing their individual datasets to one another. The ability to perform confidential queries in a distributed

manner enhances trust among stakeholders, fosters data sharing, and ultimately drives innovation and insights that were previously unattainable due to privacy concerns. In essence, DCQP for Spark is motivated by the dual goals of maximizing the utility of distributed data processing frameworks and adhering to the highest standards of data privacy and security.

1.1.3 Encryption Techniques

DCQP uses numerous encryption strategies to ensure the confidentiality of facts during the processing of the request. It consists of isomorphic encryption, stable multiparty computation (SMC), and various cryptographic primitives. These strategies can calculate directly on the encrypted facts without the need for encryption, for this reason they protect the privacy of the information.

1.1.4 Distributed Query Processing

DCQP extends the question processing skills of Apache Spark to manual allotted execution of encrypted queries. It integrates with Spark's execution engine to permit regular computation across multiple nodes in the cluster whilst retaining data confidentiality[2]. This permits groups to leverage the scalability and common overall performance blessings of Spark for processing sensitive records.

1.2 Core Components

1.2.1 Query Executor

Spark is responsible for running searches on encrypted logs within the cluster Cross-aggregation coordinates query responsibilities between nodes while ensuring data privacy.

1.2.2 Encryption Level

Provides a mechanism to encrypt and decrypt statistics as you complete the query processing process. This layer integrates with Spark's data processing component to enable retrieval computation of encrypted statistics.

1.2.3 Key Management

Manage encryption keys used to mask real-time facts and perform cryptographic operations on the system. Ensures key distribution, rotation and method checks are met to maintain data security.

1.2.4 Secure Communications

A Spark implements a protocol for resuming calls between nodes within the fold that preserves eavesdropping and statistical constraints. This ensures that information remains private during delivery between clean parts of the device.

1.3 Use Cases

The Distributed Confidential Query Protocol (DCQP) can be applied across various domains where data privacy is paramount, facilitating critical data analytics while ensuring confidentiality. Key use cases include:

- *Health Research:* In healthcare, patient data is highly sensitive and protected by regulations like HIPAA. DCQP allows researchers to perform comprehensive analyses on medical records from multiple hospitals without exposing individual patient information. This capability is crucial for studies involving large datasets, such as epidemiological research, where combining data from different sources enhances the reliability and scope of findings while maintaining patient confidentiality.
- *Economic Research:* Economists often require access to proprietary financial data from different institutions to build accurate economic models and forecasts. With DCQP, financial institutions can contribute their data to collective analyses without disclosing sensitive information, ensuring compliance with privacy laws like GDPR. This fosters collaboration among institutions, leading to more robust economic insights and policy recommendations.

- *Public Information Sharing:* Government agencies and public organizations frequently need to share data with researchers and other agencies while protecting individual privacy. DCQP enables these entities to perform joint analyses on public data without compromising privacy. For example, during a public health crisis, various agencies can share data to track the spread of a disease and evaluate the effectiveness of interventions, all while ensuring the confidentiality of personal data.

By enabling secure and confidential query processing, DCQP helps organizations comply with regulatory requirements and protect sensitive information from unauthorized access. This protocol not only enhances data security but also promotes collaborative research and innovation by allowing safe data sharing across organizational boundaries. Consequently, DCQP is instrumental in leveraging the full potential of distributed data analytics while upholding the highest standards of data privacy.

Chapter Two Preliminaries

2. 1 Distribute System

A Distributed System is a collection of autonomous computer systems that are physically separated but are connected by a centralized computer network that is equipped with distributed system software[3]. The autonomous computers communicate among each system by sharing resources and files and performing the tasks assigned to them. Distributed network design is one of the most graphical representations of intelligent systems that are capable of independently computing “low-level” tasks to a high standard and working together while release themselves from centralized authority. In this architecture each of node is on its own, yet trying to keep them working on unanimous goals. Decentralization not just a design principle but a core building block. It brings together concurrency, supreme flexibility, scalability, and fault tolerance in a tightly knit architecture for maximum efficiency[4]. Message transfer is performed across the network nodes over connections designed to guarantee consistency and replication. In particular, the data integrity is maintained using the deep understanding of key concepts. Security norms such as encryption and access control that deny hackers any legal foothold to connected networks would be faster at stopping attacks. Disseminated systems of different nature are all over the modern world that make peer-to-peer communication as well as cloud processing and data processing possible and evidence continuous intelligence of the humanity in the computer world[5]. Here are some key aspects of distributed systems: 1. Decentralization, 2. Concurrency, 3. Scalability, 4. Fault Tolerance, 5. Communication, 6. Consistency and Replication, 7. Security

2.2 Confidential Query Protocol

The Confidential Query Protocol (CQP) is an encryption employed in distributed systems to allow the transfer of confidential information while guaranteeing against

unauthorized access into remotely located data. It enables the performance of queries on non-apologies globally distributed databases or data storage systems in a way that preserves the privacy and the right to be forgotten of the data of the individuals. Key features of the Confidential Query Protocol include:

- *End-to-End Encryption:* CQP utilizes message encryption technique (queries and responses) thus ensuring that the security attacks such as query tampering do not happen. CQP strategy is ensured information exchange during different phases.
- *Data Confidentiality:* The protocol make data useless for any intermediate nodes and also the protocol ensures the secure move of data without compromising the confidentiality.
- *Query Integrity:* By CQP it is possible to refute the request from users with malintention and to approve them with the given level of authorization.
- *Access Control:* It employs policies of authentication, giving access only to the ones with sufficient approval for requisite queries and writing data.
- *Authentication:* CQP is how to identify the person who made a request of query and provider of (data/information), thus only the authentic entities will have the access of the query.
- *Privacy-Preserving Techniques:* It is also accompanied by the manifestation of these approaches such as homomorphic encryption; or secure multi-party computation, among others.

At last, besides the data confidentiality being secured while querying, organizations are able to benefit from the distributed storage without need for taking security measures.

2.3 Spark

Apache Spark is an open-source, distributed processing system used for big data workloads[6]. It provides a faster and more general data processing platform[7]. In 2009,

Apache Spark released as a research project at UC Berkeley's AMPLab aimed at making parallel programming easy through an abstraction layer with ease of use, support and portability of datasets, with a focus and investigation on data-intensive application domains. Spark was built to improve on the current Hadoop MapReduce by developing a new framework that has speed capacities, can do loop iterations, and be interactive for data analysis with having the attributes of the Hadoop MapReduce. The first paper entitled, “Spark: “Cluster Computing with Working Sets” that featured Spark was published on June 2010, and on the same day year, Spark is under the BSD license. Since June, 2013, Spark became Apache Incubation project on Apache Software Foundation and finally Spark reached Apache top-level project status in February, 2014. Spark can be used in an isolated way, on the top of Mesos, or more frequently in Hadoop platform.

Apache Spark involves the executing of jobs using an available distributed resource pool consisting of Resilient distributed datastores for processing preparation of data[8]. Cluster driver program that a driver program of the driver program does tasks with the nodes of the nodes where a cluster manager does execute this. Worker nodes that are running on these machines process tasks on RDD partitions in parallel. This decreases the time it takes to run calculations since it prevents the data retrieval from being a bottleneck to passing the data to memory. If data gets lost somehow during the hull line, the system can re-calculate and restore those lost parts. It was creating a place for SQL, Streaming, ML and Graph Processing separate engines does not provide unified platform so Spark was created as the solution to this problem. Basically speaking, Spark approaches this problem in a different way than other systems, using RDD, deficient tolerance, and a set of libraries that provide comprehensive handling of data processing tasks regardless of whether they are distributed or not. Apache Spark is a distributed

computing system, designed to run on Resilient Distributed Datasets (RDDs). Some ideas include:

- *Driver Program:* One of the principal features of the code is breaking it down to the cluster via creating the tasks and their components.
- *Cluster Manager:* Directs the group of resources and then move them towards where the applications that are running on the Spark deployments are situated.
- *Worker Nodes:* The driver program is designed to perform programmable tasks on the machines of the cluster; this aims to accomplish the driver operation prompt.
- *Task Execution:* There is a job queued for execution on each RDD partition. Every variant has its own responsibility to transform or do some action defined in the job.
- *In-Memory Computation:* Through lined up in-memory storage as a leading element, the data process speed will be catapulted.
- *Lazy Evaluation:* Transformations get placed deep into the DAG; triggers action signals the threads function completion.
- *Fault Tolerance:* While RDD Lineage provides the function of recovering the lost partitions, it is also able to go back and recalculate the missing partitions.
- *Integration:* Spark contains a package of multi-format libraries that facilitate SQL, Streaming, ML, and graph processing.

The Spark eco-fame lends its support to a ruthless distributed model that comes with some unique features and libraries, and this is the reason it is the favorite operating system that many people use in organizations as it excels in file processing of large data sets.

2.4 Design and Implementation of Spark Architecture

The shining sun around this architecture is how it's design and implementation are guided that has ability to process a lot of data/instruction while sustaining fault tolerance and

not tiring servers. At the heart of Spark, RDDs or Resilient Distributed Dataset is the primary abstraction that carries the raw data. The architecture consists of several key components:

- *Driver Program:* The driver program in this Catalyst is the core coordinator of the Spark app execution. It interacts with a manager node for resources providing and decides on which slave nodes will run set of tasks.
- *Cluster Manager:* Cluster manager's responsibility consists in resource management and distributing them over the cluster when Spark applications try to use them. A cluster manager can be built-in, e.g., Spark has its own cluster manager, or it can be used with other frameworks like Apache Mesos or Hadoop YARN that also manage clusters. YARN is composed of Resource Manager and Node Manager[9].
- *Worker Nodes:* Nodes of a leanness are the machines where active tasks are carried out. E.g., each worker process area, and in turn, every executor process cares for task execution & stores the data either in memory or some disk.
- *Resilient Distributed Datasets (RDDs):* RDD is a data abstraction of basic type that generally is used in Spark. They are the sorts of storage media that are very dependable and good for writing data on that can be worked on at the same time. RDDs enable transformation operations, which in turn, generate new RDDs, and actions which initiate computation and the returns the results.
- *In-Memory Computation:* The power of Spark's in-memory computation allows for smoother performance[10]. It does it by holding up interim data in memory (or disk if memory won't suffice) and recursively writing it to further computing, therefore skipping frequently disk reading.
- *Fault Tolerance:* Spark guarantees endurance using RDD lineage. Through keeping the lineage of each RDD in record thus Spark can reconstruct the lost partition in any node failure scenarios.

- *Integration with Libraries:* Spark can join and combine SQL libraries (Spark SQL), streaming (Spark Streaming), machine learning (MLlib), and graph processing operating devices (GraphX), allowing the use of a platform tool for data processing from a unified platform[11].

An important part of Spark architecture ensures that the system is efficient, fault-resilient, and simple to use. Therefore, thanks to that, the framework is able to perform various operations on the data sets of any scale in parallel computer environments. The RDD programming model pro-vides only distributed collections of objects and functions to run on them[12].

Chapter Three Architecture Design

3.1 Problem Description and Design Philosophy

3.1.1 Problem Description and Overall Design Concept

In the era of big data, organizations increasingly rely on distributed computing frameworks like Apache Spark to process and analyze vast datasets. However, these datasets often contain sensitive information, such as personal health records, financial data, and proprietary business information. Ensuring data confidentiality during distributed processing poses significant challenges. Traditional security measures are inadequate because they do not protect data during computation and may not comply with stringent data privacy regulations such as GDPR and HIPAA. The primary problems that need to be addressed are:

- *Data Privacy:* Ensuring that sensitive data remains confidential during distributed processing.
- *Regulatory Compliance:* Adhering to data privacy laws and regulations while performing complex data analytics.
- *Secure Multi-party Computation:* Enabling multiple parties to collaboratively analyze data without revealing their individual datasets to one another.

The Distributed Confidential Query Protocol (DCQP) is designed to address these challenges by incorporating advanced cryptographic techniques into Spark's distributed processing framework. The overall design concept includes the following components:

- *Data Encryption:* All sensitive data is encrypted before it is loaded into the Spark environment. Encryption ensures that data remains confidential even if the computing infrastructure is compromised.
- *2. Secure Query Processing:* DCQP modifies Spark's query processing engine to handle encrypted data. It employs techniques such as homomorphic encryption and secure multi-party computation to perform operations on encrypted data without decrypting it.

- *3. Privacy-Preserving Data Sharing:* The protocol allows multiple organizations to share and analyze their data collectively. DCQP ensures that individual datasets remain confidential and are only combined at an encrypted level. Results are only decrypted when the final analysis is complete and only by authorized parties.
- *4. Regulatory Compliance:* DCQP includes features to ensure compliance with data privacy regulations. It provides auditing capabilities to track data access and processing activities, ensuring transparency and accountability.
- 5. Scalability and Performance: While ensuring security and confidentiality, DCQP is designed to leverage Spark's inherent scalability and performance. Optimizations are included to minimize the overhead introduced by encryption and secure computation.
- *6. User-Friendly API:* DCQP provides a user-friendly API that integrates seamlessly with existing Spark workflows. This ensures that users can easily adopt the protocol without requiring extensive changes to their current processes.

By integrating these components, DCQP provides a robust solution for secure and confidential data processing in distributed computing environments. It empowers organizations to perform complex data analytics while maintaining the highest standards of data privacy and regulatory compliance.

3.1.2 Design Philosophy of The System Architecture

The design philosophy of the Distributed Confidential Query Protocol (DCQP) system architecture is centered around ensuring data confidentiality, regulatory compliance, and efficient performance in distributed computing environments. The following principles guide the architecture design:

- *Security by Design:* Security is not an afterthought but an integral part of the system architecture. DCQP incorporates advanced cryptographic techniques from the ground up,

ensuring that data is always encrypted during storage, transit, and computation. This approach minimizes the risk of data breaches and unauthorized access.

- *Data Privacy Preservation:* The architecture is designed to preserve data privacy at all stages of processing. Techniques like homomorphic encryption and secure multi-party computation are employed to enable computations on encrypted data. This ensures that sensitive information is never exposed in plaintext, even during complex analytical tasks.
- *Regulatory Compliance:* Compliance with data privacy regulations such as GDPR and HIPAA is a core consideration. The system includes features for data auditing, access control, and data usage tracking, ensuring that all data processing activities can be monitored and verified for compliance purposes.
- *Scalability and Performance:* The architecture leverages Spark's distributed computing capabilities to ensure scalability and high performance. The design optimizes for minimal overhead introduced by encryption and secure computation, ensuring that the system can handle large-scale data processing tasks efficiently.
- *Modularity and Extensibility:* The system is designed with a modular architecture, allowing different components to be developed, updated, or replaced independently. This modularity facilitates the integration of new cryptographic techniques and performance optimizations as they become available.
- *Transparency and Auditability:* Transparency in data processing is crucial for trust and compliance. The system architecture includes comprehensive logging and auditing capabilities, allowing stakeholders to trace data processing activities and verify that privacy-preserving measures are being applied correctly.
- *Ease of Integration:* The system provides a user-friendly API that integrates seamlessly with existing Spark workflows. This ensures that organizations can adopt DCQP without

significant changes to their current data processing pipelines, reducing the barrier to entry and facilitating widespread adoption.

- *Collaboration Facilitation:* The architecture supports secure multi-party computation, enabling different organizations to collaboratively analyze data without revealing their individual datasets. This fosters data sharing and collaborative analytics while maintaining strict confidentiality.
- *User Control and Consent:* Users retain control over their data, with mechanisms in place to ensure that data processing only occurs with explicit consent. This aligns with ethical considerations and legal requirements for data usage.

By adhering to these design principles, the DCQP system architecture ensures that data processing in distributed environments is secure, compliant, and efficient, enabling organizations to leverage the full potential of big data analytics while safeguarding sensitive information.

3.1.3 Deployment Diagram of Architecture Design

The diagram illustrating overall the architecture and workflow of a distributed computing system of spark.

- The diagram in the below shows the main node, which contains two processes: “Primary” and “Driver application.main()”.
- Three worker nodes are connected to the master node, each labeled "Worker Node 1", "Worker Node 2" and "Worker Node 3".
- We can see a “Worker” process connected to an “ExecutorRunner” in each worker node.
- Below the diagram we see “ExecutorRunner”, this is a “CoarseGrainedExecutorBackend” process that controls several “Executor” objects.
- These executors manage tasks, represented by wavy lines. The arrows in the diagram indicate the communication flow between different nodes and processes.

- In the lower right corner, there is a legend explaining that the rectangles represent processes and the ovals represent objects.

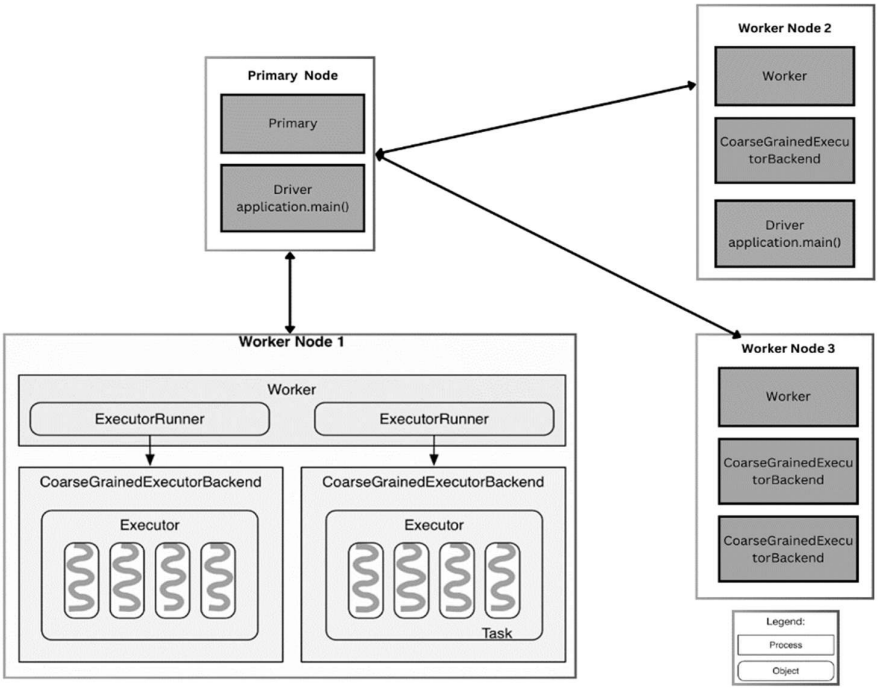


Figure 1- 3.1.3 Overall the architecture and workflow

This diagram illustrates Spark, a distributed computing system. In Spark, the primary node executes the driver program, while the worker nodes handle the executor tasks. Executors are responsible for running tasks that comprise your Spark application. The CoarseGrainedExecutorBackend component of Spark is crucial as it manages the lifecycle of these executors. This involves initializing, monitoring, and terminating executors to ensure efficient task execution within the Spark framework. This architecture allows Spark to efficiently distribute and process large-scale data workloads across a cluster of machines, ensuring high performance and scalability.

3.2 Systematic Structure

The process by which the driver application on the primary node completes its task and then sends it to the worker nodes on Spark is shown in the diagram below:

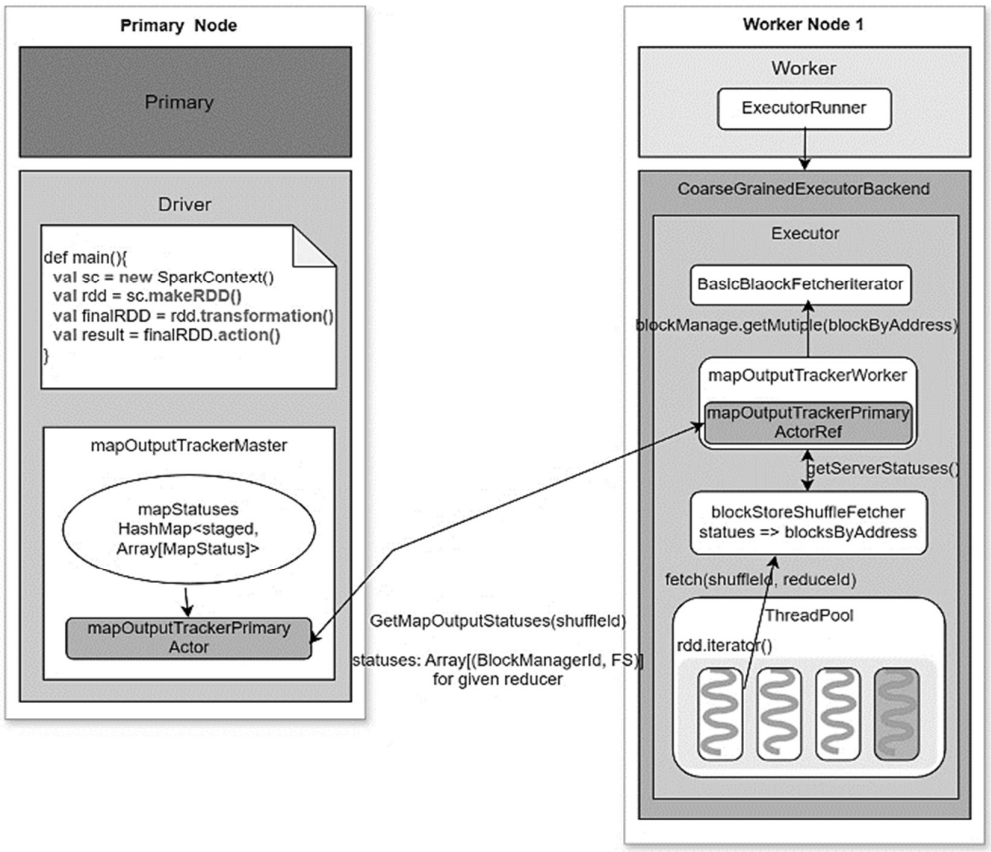


Figure 2- 3.2 The driver application

The following code represents driver-side behavior:

```
// Execute the final action on the RDD, triggering computation in the Spark Context.
finalRDD.action() => sc.runJob()

// Assemble the job and its components.
=> dagScheduler.runJob()
=> dagScheduler.submitJob()
=> dagSchedulerEventProcessActor ! JobSubmitted
=> dagSchedulerEventProcessActor.JobSubmitted()
=> dagScheduler.handleJobSubmitted()
=> finalStage = newStage()
=> maputputTracker.registerShuffle(shuffleId, rdd.partitions.size)
=> dagScheduler.submitStage()
=> missingStages = dagScheduler.getMissingParentStages()
=> dagScheduler.submitMissingTasks(readyStage)

// Submit the tasks for execution and manage their scheduling.
=> taskScheduler.submitTasks(new TaskSet(tasks))
=> fifoSchedulableBuilder.addTaskSetManager(taskSet)

// Dispatch tasks to available executors.
=> sparkDeploySchedulerBackend.reviveOffers()
=> driverActor ! ReviveOffers
=> sparkDeploySchedulerBackend.makeOffers()
=> sparkDeploySchedulerBackend.launchTasks()
```

```
// For each task, initiate its execution on an executor.
=> foreach task
  CoarseGrainedExecutorBackend(executorId) ! LaunchTask(serializedTask)
```

This driver program's environment is built up by initializing the *SparkContext* val sc = new SparkContext(sparkConf), which defines its function as the coordinator of the Spark application. It creates the settings and channels of communication required to communicate with the cluster.

3.3 Logical Plan

transformation() to create a calculation chain (RDD sequence) in the driver software. Inside each RDD: *compute()* functions specify how to count entries for their partitions. *getDependencies()* dependencies between RDD partitions are defined by functions.

3.4 Physical Plan

Every *action()* starts a task: *DagScheduler.runJob()* has several stated steps. In *submitStage()*, *ShuffleMap* and *ResultTasksOnce* the tasks required by the stage are completed, they are bundled into *TaskSets* and forwarded to *TaskScheduler*. Tasks will be sent to *sparkDeploySchedulerBackend* for task distribution if *TaskSet* is able to be executed.

3.5 Distribution

Upon receiving *TaskSet* from *sparkDeploySchedulerBackend*, the Driver Actor forwards serialized tasks to the *CoarseGrainedExecutorBackend* Actor on the worker node.

3.6 Reception

In the program the executor selects a free thread to execute each job after packaging it into *taskRunner*. An executor is the only one in a *CoarseGrainedExecutorBackend* process.

The following will be done by the worker after getting tasks:

```
// Initiate the task launch request to the executor backend.
=> coarseGrainedExecutorBackend.launchTask(serializedTask)

// Within the executor, process the task launch request.
=> executor.handleTaskLaunchRequest()

// Execute the task within the executor's thread pool.
=> executor.threadPool.execute(new TaskRunner(taskId, serializedTask))
```

3.7 Task Execution

I present an example of *task-execution* so that we can understand how the worker nodes and principal nodes interact in the spark system. An ensuing task is sent to the executor, who then parse it into a standard task, runs the job to get the *directResult*, and returns the result to the driver.

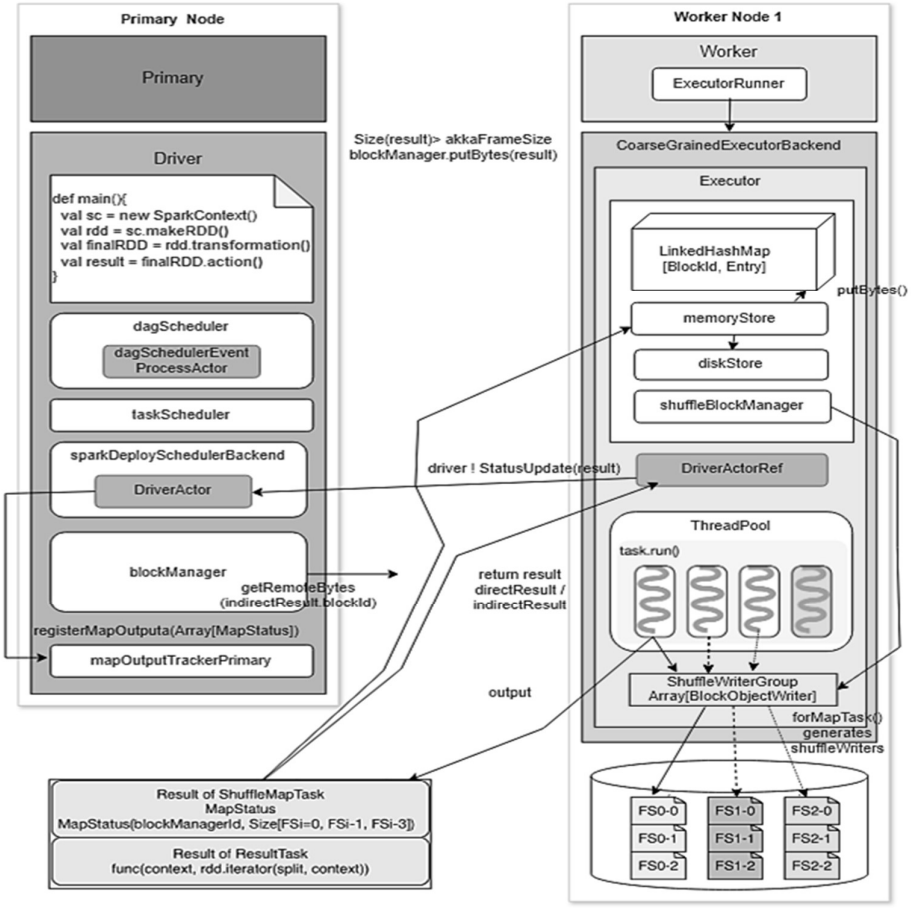


Figure 3- 3.7 Task-Execution

It is important to remember that the *Actor* data package size cannot exceed a certain limit: The result will be handled by *blockManager* and persisted to "memory + hard disk" if it is very large (such as the *groupByKey* result). The storage location is the only indirect result that the driver will receive. When necessary, the driver will use HTTP to retrieve the outcome. Upon receiving a result that is sufficiently little (below *spark.akka.frameSize*, which is usually 10MB), the driver will immediately obtain it. Additional information on *blockManager*: *blockManagers memoryStore* uses a *LinkedHashMap* to save the data it holds in memory, so that the map does not take up more space than the available memory, when the result is larger than *akka.frameSize* times the *spark.storage.memoryFraction* times the *getRuntime.maxMemory* (default 0.6). If the data storage level is set to "disk" and the *LinkedHashMap* is full, the incoming data will be sent to *diskStore*, which saves the data on a hard drive.

```
In TaskRunner.run():
// Deserialize task, execute it, and then send the result to the executor backend for status update.
=> task = ser.deserialize(serializedTask)
=> value = task.run(taskId)
=> directResult = new DirectTaskResult(ser.serialize(value))
=> if (directResult.size() > akkaFrameSize())
     indirectResult = blockManager.putBytes(taskId, directResult, MEMORY+DISK+SER)
  else
     return directResult
// Inform the executor backend about the task execution status update.
=> coarseGrainedExecutorBackend.statusUpdate(result)
=> driver ! StatusUpdate(executorId, taskId, result)
```

The outcomes of *ShuffleMapTask* and *ResultTask* are different. *MapStatus* is a result of *ShuffleMapTask*, which is divided into two parts: the tasks *BlockManagers BlockManagerId*, which is the combination of the executors ID, host, port, and *nettyPort,* and the output size for each Section of a task file. *ResultTask* produces the functions execution result on a single partition. *Count()* is a function that counts the number of records in a partition. Writers from *OutputStream* are required because *ShuffleMapTask* requires *File_Segment* in order to write data to disk. Shuffle's *blockManager* is in charge of producing and overseeing these *authors.BlockManager*.

```
In task.run(taskId):
// Execute the task based on its type
=> if (task is ShuffleMapTask)
    return shuffleMapTask.runTask(context)
      => shuffleWriterGroup = shuffleBlockManager.forMapTask(shuffleId, partitionId, numOutputSplits)
      => shuffleWriterGroup.writers(bucketId).write(rdd.iterator(split, context))
      => return MapStatus(blockManager.blockManagerId, Array[compressedSize(File_Segment)])
  else if (task is ResultTask)
    return func(context, rdd.iterator(split, context))
```

The driver will carry out the following actions after receiving the task's result. After the job is completed, *TaskScheduler* will be alerted and the result will be processed: The real results will be retrieved by calling *BlockManager.getRemotedBytes()* if the result type is *indirectResult. ResultHandler()* calls driver-side computation on the result if it *is ResultTask* (*count()* takes the sum operation on all *ResultTask,* for example). If the *MapStatus* comes from the *ShuffleMapTask,* it will be added to the *mapStatuses* of the *mapOutputTrackerMaster,* making it easier to query throughout the decrease shuffle process. If the drivers last job was the last one in the stage, they will submit the next step. If this is the last stage, *dagScheduler* will be informed that the work is finished.

```
After the driver receives StatusUpdate(result):
=> taskScheduler.statusUpdate(taskId, state, result.value)
=> taskResultGetter.enqueueSuccessfulTask(taskSet, tid, result)
// Handle the result based on its type
=> if (result is IndirectResult)
    serializedTaskResult = blockManager.getRemoteBytes(IndirectResult.blockId)
=> scheduler.handleSuccessfulTask(taskSetManager, tid, result)
=> taskSetManager.handleSuccessfulTask(tid, taskResult)
=> dagScheduler.taskEnded(result.value, result.accumUpdates)
=> dagSchedulerEventProcessActor ! CompletionEvent(result, accumUpdates)
=> dagScheduler.handleTaskCompletion(completion)
=> Accumulators.add(event.accumUpdates)
// Handle ResultTask completion
=> if (job.numFinished == job.numPartitions)
    listenerBus.post(SparkListenerJobEnd(job.jobId, JobSucceeded))
// Notify listeners and handlers for successful completion
  => job.listener.taskSucceeded(outputId, result)
  => jobWaiter.taskSucceeded(index, result)
  => resultHandler(index, result)
// Handle ShuffleMapTask completion
=> stage.addOutputLoc(smt.partitionId, status)
=> if (all tasks in the current stage have finished)
// Register map outputs and submit the stage for further processing
    mapOutputTrackerMaster.registerMapOutputs(shuffleId, Array[MapStatus])
    mapStatuses.put(shuffleId, Array[MapStatus]() ++ statuses)
=> submitStage(stage)
```

3.8 Shuffle

We discussed *task-execution* and result processing in the paragraph above. In this paragraph, we'll discuss how the reducer (tasks requires shuffle) obtains the input data: The *File_Segments* generated by *ShuffleMapTask* of the parent stage must be identified to the reducer on whose node they are. When *ShuffleMapTask* is completed, the driver's *mapOutputTrackerMaster* receives this kind of data. Additionally, the data is stored into *mapStatuses* which is: *HashMap [mapStatus*, *Array[stageId]]*. Also, we could get an *Array [MapStatus]* with *stageId*, which gives us information about *File_Segments* created by *ShuffleMapTasks*. The array *(taskId)* contains the *blockManagerId* and the size of each *File_Segment*. In order to determine the location of the input data *(File_Segments),* reducer will first call *blockStoreShuffleFetcher* when it needs to retrieve input data. To complete the task, *blockStoreShuffleFetcher* invokes local *MapOutputTrackerWorker.* In order to connect with *mapOutputTrackerMasterActor* and obtain *MapStatus, MapOutputTrackerWorker* utilizes *mapOutputTrackerMasterActorRef.* After processing *MapStatus, blockStoreShuffleFetcher* determines where the reducer should retrieve *File_Segment* data, which is then stored in *blocksByAddress.* To get *File_Segment* data, *blockStoreShuffleFetcher* instructs *basicBlockFetcherIterator.*

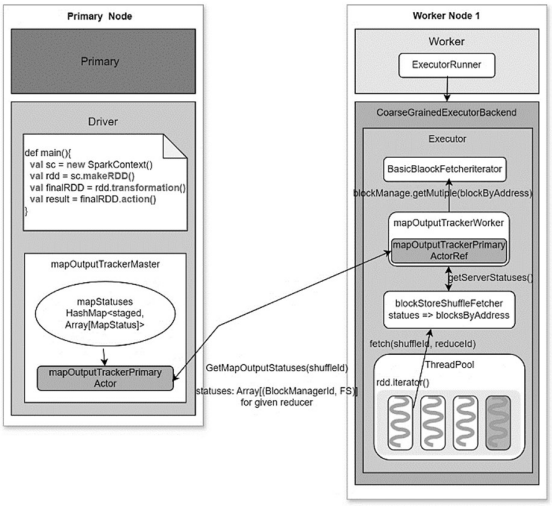


Figure 4- 3.8 Shuffle process

Following receipt of the data retrieval job, *basicBlockFetcherIterator* generates several *fetchRequests*. "The tasks to get *File_Segments* from many nodes are contained in each of them." We may additionally infer from the above diagram that reducer-2 should retrieve File_Segment this is (FS) into (in white) from three worker nodes.

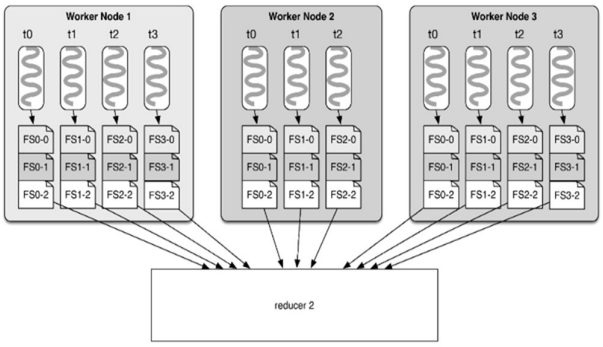


Figure 5- 3.8 File Segments

In the *blockByAddress* function may be used to represent the global data fetching task: Blocks four from node one, three from node two, and four from node three. To optimize the data fetching process, it seems reasonable to break up the global jobs into smaller tasks *(fetchRequest),* and then assign a thread to each task to get data. Similar to Hadoop, Spark starts five parallel threads for every reducer. One fetch is as it were permitted to collect as much information as *spark.reducer.maxMbInFlight = 48MB* since the gotten information will be buffered into memory. Each subtask ought to not take longer than 48MB / 5 = 9.6MB, as each of the five bring strings offers 48MB. We ought to break between t1_r2 and t2_r2 since, concurring to the diagram, size (FS0_2) + size (FS1_2) < 9.6MB on node-1 and size (FS0_2) + size(FS1_2) + size(FS2_2) > 9.6MB. Thus, two *fetch_Requests* that are recovering information from node-1 are unmistakable. We can assume a *fetch_Request* will be greater than (>) 9.6MB and will it be made accessible? The response should be "Yes". Even if a *File_Segment* is too big, it must still be fetched all at once. Additionally, reducer will do a local read if it requires any *File_Segments* that are currently present on the local. It will deserialize the *File_Segment* that was fetched at the end of the shuffle read and provide record iterators to

RDD.compute(). A few additional specifics: How is the *fetchRequest* sent to the destination node by the reducer? How is the File_Segment read, processed and returned to the reducer by the target node?

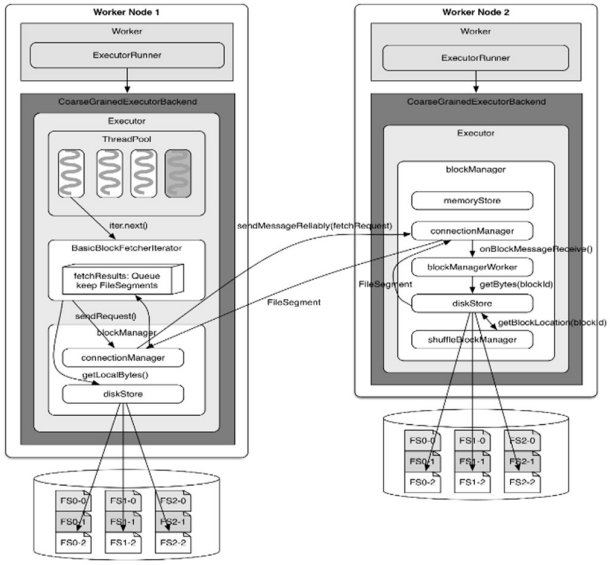


Figure 6- 3.8 Fetch Request process

File_Segments will be fetched by calling *BasicBlockFetcherIterator* when *RDD.iterator()* satisfies *ShuffleDependency*. For sending *fetch_Request* to the connectionManagers which is necessary to the other nodes, *BasicBlockFetcherIterator* uses *connectionManager* of *blockManger. ConnectionManagers* communicate with one another over NIO. For instance, on the other nodes, *blockManager* will get a message from *connectionManager* on worker node 2 once it is received. In the latter case, *File_Segments* requested by *fetchRequest* are read locally using *diskStore; connectionManager* will still send them back. In the event that *FileConsolidation* is enabled, *diskStore* requires the *blockId* location provided by *shuffleBolockManager.* When reading a file segment, *diskStore* will place it in memory if its size is less than *spark.storage.memoryMapThreshold* = 8KB; if not, large file segments can be loaded since the memory mapping method in *FileChannel* of *RandomAccessFile* will be utilized to read the file segment. When it receives the serialized *File_Segments* from the other nodes, *BasicBlockFetcherIterator* will deserialize and store them in *fetch_Results*. As

fetch_Results shows you. Queue is similar to the *softBuffe*r in the shuffle section. If *diskStore* finds the (FS)*File_Segments* locally and includes them in *fetch_Results*, that is, if they are required *by BasicBlockFetcherIterator*. After that, Reducer then reads and processes the records from *File_Segment*. There is a *BasicBlockFetcherIterator* for each reducer, and theoretically, one *BasicBlockFetcherIterator* could contain 48MB of *fetch_Results*. As soon as one *(FS)File_Segment* in *fetch_Results* is read off, several more File_Segments will be retrieved to fill that 48MB[13].

```
In BasicBlockFetcherIterator.next():
result = results.task()

// Process fetch requests while ensuring the maximum bytes in flight limit is not exceeded
while (!fetch_Requests.isEmpty &&
       (bytesInFlight == 0 || bytesInFlight + fetch_Requests.front.size <= maxBytesInFlight)) {
    sendRequest(fetch_Requests.dequeue())
  }

// Deserialize the fetched result
result.deserialize()
```

Chapter Four Prototype Implementation

4.1 Overall Concept Logical Scheme

The existing Spark system runs in a master-slave mode. We need to create the initial RDD from any related sources using such methods like data structures stored in RAM, local files, HDFS, and HBase. Keep in mind that *parallelize()* and *createRDD()* are comparable. Many RDD transformation procedures are grouped together under the *transformation()* label. One or more serial RDD[T]s are processed in each *transformation()*, where T is a type that can be used in any Scala program. If T is (K, V), it cannot be converted to an accumulation form (Array, List, etc.) since it is hard to implement the *partition()* on collections.

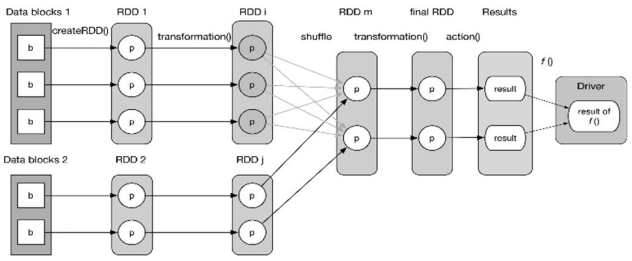


Figure 7- 4.1 RDD Workflows

When actions occur this is called by the RDDs as *action()* for final RDD. Partition sizes and the type of program, however, can either speed up or slow down computation of result. As the result, at the period of action operation the action function is executed for the final RDD. Following that, they give a series of operations and get the computing results on the terminal. After the driver receives these results, the result will be calculated using "*f (List [Result])"*. For example, *count()* requires two stages: action() and sum(). Here, we can see that by using the *checkpoint(), cache() or persist()* functions, the RDD may be stored in memory or persist on disk. Additionally, the user often determines the number of partitions. There are two possible partition mapping configurations for two "*RDDs: 1:1 and M : N"*. The provided image allows us to see both "*1:1 and M : N"* mapping. In broad terms, additional RDDs would be formed throughout the runtime, even though I had a fair sense of how many would be created, for

example, when I initially started writing Spark code. An un-computed block is passed as agitator that can be looped through to compute the values of the RDD block one by one[14]. In order to clarify my logical plan, we can think of questions that can sum up the expected result, such as how to produce RDDs and what kind of RDDs might be produced. How do I connect RDD to build the data dependency? While a *transformation()* typically creates a single RDD, certain transformation()s have the ability to create many RDDs due to their various processing stages or several sub-*transformation()*s[5]. Because of this, there are really more RDDs than we initially estimated. A logical plan is just a succession of computations. Every RDD utilizes the *compute()* function that takes in records to be processed (key/value combinations, for instance) obtained from the prior RDD or data source, uses transformation() to process the records, and then produces new records. Different RDDs should be produced depending on the computational logic of the transformation. Let's now talk about a few popular *transforms()* and the RDDs they produce. On the Spark website, we can find out what each *transformation()*means[5]. More details are shown in the subsequent table, in which iterator(split) indicates for every entry within the segment. The sophisticated *transformations()* that result in many RDDs are the reason for some of the blanks in the table[13, 15].

Transformation()	Generated RDDs	Compute()
map(func)	MappedRDD	iterator(split).map(f)
filter(func)	FilteredRDD	iterator(split).filter(f)
flatMap(func)	FlatMappedRDD	iterator(split).flatMap(f)
mapPartitions(func)	MapPartitionsRDD	f(iterator(split))
mapPartitionsWithIndex(func)	MapPartitionsRDD	f(split.index, iterator(split))
sample(withReplacement, fraction, seed)	PartitionwiseSampledRDD	PoissonSampler.sample(iterator(split)) BernoulliSampler.sample(iterator(split))
pipe(command, [envVars])	PipedRDD	
union(otherDataset)		
intersection(otherDataset)		
distinct([numTasks]))		
groupByKey([numTasks])		
reduceByKey(func, [numTasks])		
sortByKey([ascending], [numTasks])		
join(otherDataset, [numTasks])		
cogroup(otherDataset, [numTasks])		
cartesian(otherDataset)		

coalesce(numPartitions)		
repartition(numPartitions)		

Table 1- 4.1 Iterator indicates for every entry within the segment

4.2 Intricate Logical Scheme

In this part, I will talk about our complex idea about the physical plan that I carried out while working on this project. Identification of all the interrelated steps which are formed into the data and realizing them is valid how? In order to create the stage, the RDD must be the next one, so it must be attached to the one that is before it. This sequence is further shown in the next arrow, which will then lead to the creation of jobs. Join these RDDs like these, and we will use stage three of them as the stage. Admittedly, it is a tactic that might work in some limited cases, but such an approach becomes of no use when other approaches to the conflict are switched to a dominant approach[16]. It has a minor but serious issue: in the overall standing, a huge portion of the data is of a transitional nature. The outcome of a physical job will be kept in memory, on the local disk, or on both. If every arrow throughout the data reliance structure produces a task, then the system has to store data from all RDDs. It will be quite expensive. A closer look at the logical scheme could reveal that each RDD's divisions are independent of one another. In other words, no data within a partition inside an RDD will interact with any other data. Given this discovery, a bold suggestion would be to treat the entire diagram as a single step and design a single physical job for every final RDD partition *(FlatMappedValuesRDD).* This concept is illustrated in the diagram below:

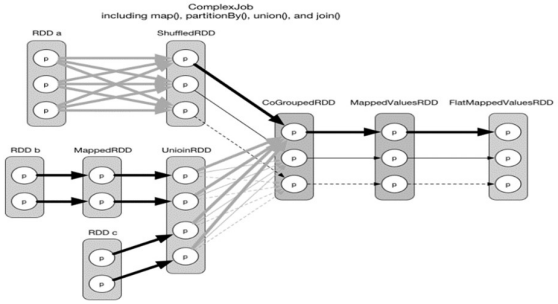


Figure 8- 4.2 RDD partition

All of the thick arrows in the preceding picture correspond to task 1, which is the initial split of the task's eventual RDD. Since it's a ShuffleDependency, and we want to determine the very first segment that comprises CoGroupedRDD, we have to compute all of the partitions of the RDDs that came before it (all of the partitions in UnionRDD, for reference). Next, we may compute the second and third partition of the CoGroupedRDD using task 2 (tiny arrows) as well as task 3 (fled arrows), respectively. These are much simpler. However, there are two problems with this idea:

- The first assignment is very big. The ShuffleDependency forces us to calculate every partition of the previous RDDs (UnionRDD).
- To decide which divisions (calculated in the first work) should be stored for the subsequent tasks, it is necessary to construct complex algorithms.

This concept, which involves pipelineing the data so that it is only computed when it is truly needed in a flow form, has merit even in spite of its flaws. For instance, in the first job, we determine which RDDs and partitions truly need to be calculated by working backwards from the end RDD (FlatMappedValuesRDD). Furthermore, there is no need to keep any intermediary data between the RDDs with NarrowDependency relation[11]. As an example, the logical plan may be categorized as the following phases:

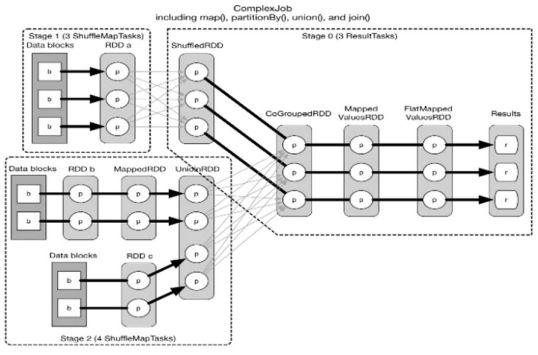


Figure 9- 4.2 The logical plan

If we examine the partition connection from a record-level perspective, the pipelining will make more sense. The primary problem with the bold notion mentioned above is that if there

is a *ShuffleDependency*, we cannot pipeline the data flow[17]. Due to the pipelineability of Narrow Dependency, users may halt the data flow at each *ShuffleDependency*, leaving chains of RDDs connected by *NarrowDependency*. The method used to generate stages is to insert each NarrowDependency into the stage that is currently in progress, work backwards from the final RDD, and then, in the event that a *ShuffleDependency* arises, break out for a new stage. The task number in each phase is determined by the partition number of the final RDD. In the above chart, each thick arrow indicates a job. Since the stages are chosen backwards, stage 0 has the ID 0 as its parent, making stages 1 and 2 its parents as well. If a task in a stage produces the final result, it is of type *ResultTask*; if not, they are of type *ShuffleMapTask*. The reason *ShuffleMapTask* obtains its name is that, like the mappers in Hadoop *MapReduce*, its results must be shuffled to the next phase. If the current stage has no parents, *ResultTask* may be thought of as a mapper, and when it receives jumbled data from its parent stages, it can be thought of as a reducer. The first *ResultTask* is represented by the thick arrows. Six *ResultTask* are formed since the step outputs the final result immediately[14]. Unlike *OneToOneDependency*, every single *ResultTask* during this process has to read two data blocks and calculate three RDDs (CartesianRDD, RDD a, and RDD b) in one task. It seems apparent that *NarrowDependency* chains may always be pipelined, regardless of the particular *NarrowDependency* form—*1:1 or N : N*, for example. The number of jobs and the number of partitions in the final RDD match. The pipeline's first pattern is similar to:

```
records.foreach { record =>
  val resultOfF = f(record)
  g(resultOfF)
}
```

We can see that no intermediate results need to be saved when we think of records as a stream. For instance, the original record1 and the result of *f(record1)* may be garbage gathered after the result of *g(resultOfF1)* is created. We can then calculate *g(resultOfF2)*. But this isn't the case for other patterns (like the third pattern, which is as follows)[13, 15]:

```
def processAndAggregate(records): Iterator[Record] = {
  val result = new ArrayBuffer[Record]() // Store the intermediate result here
  for (record <- records)
    result += process(record) // Aggregate processed records
  result.iterator // Return the iterator of the newly-generated records
}

val fResult = processAndAggregate(records) // Newly-generated records

fResult.foreach { record =>
  g(record)
}
```

4.3 Carrying out the Physical Plan

Let's return to the application's physical layout. Remember that with Hadoop MapReduce, the jobs are performed sequentially, and that *map()* produces map outputs that are written to the local disk and partitioned. Then, to create reduced inputs, namely *<k, list(v)> records*, the shuffle-sort-aggregate process is applied. The final result is generated using *reduce()*. The figure that follows provides an illustration of this process:

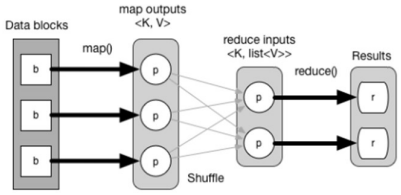


Figure 9- 4.2 Physical layout

This execution process cannot be used effectively upon Spark's physical plan as Hadoop MapReduce has a fixed, straightforward physical plan without pipelining. The main idea behind pipelining is that data is only computed in a flow-like fashion when necessary. We start with the end result and go backwards down the RDD chain to identify which RDDs and partitions are needed for obtaining the final outcome. Usually, the partitions we compute initially and trace back to are those on the leftmost RDD. When a stage has no parent stages, its leftmost RDD can be evaluated independently (which is without dependence), and each evaluated record can be piped into the subsequent computations. The actual execution streams the records ahead, while the calculation chain is inferred backwards from the last step. Before

the following record's computation begins, the previous record must complete the whole computation chain. One of the most efficient ways to achieve this is to build multiple oracle servers and make them continuously running even when an oracle server fails[18]. When a stage has parent stages, we must first complete the parent stages in order to get the data using shuffle. After completion, the situation is the same as if there were no parent stages. The data dependence of each RDD is declared in the code via its *getDependency()* function. The *calculate()* function is responsible for applying computation logic and obtaining records from the parent RDD or data sources that are downstream[17]. RDDs frequently use this kind of code: *firstParent [T].iterator (split, context).map (f).* In this instance, the first *dependentRDD* is *firstParent; iterator()* shows how the records are consumed one at a time; map(f) applies the computation logic f to each record. The *calculate()* function returns an iterative list of the calculated entries associated with this RDD, which may be used in other calculations. Thus far, the synopsis Reversing the data *depency* from the previous RDD creates the whole computation chain. Staging is done using *ShuffleDependency*. The upstream record stream is obtained at each step by calling *parentRDD.iterator()* from the *compute()* function of each RDD. Keep in mind that only computation logic is intended for use with the *compute()* function. The actual dependent RDDs are specified in dependence and the associated segments are specified in the *getDependency()* method. The approach called *getParents().* Let's analyze the Cartesian RDD as an example[13].

```
// RDD x is the cartesian product of RDD a and RDD b
// RDD x = (RDD a).cartesian(RDD b)
// Defines how many partitions RDD x should have, and the type of each partition
override def getPartitions: Array[Partition] = {
  val array = new Array[Partition](rdd1.partitions.size * rdd2.partitions.size)
  for (s1 <- rdd1.partitions; s2 <- rdd2.partitions) {
    val idx = s1.index * numPartitionsInRdd2 + s2.index
    array(idx) = new CartesianPartition(idx, rdd1, rdd2, s1.index, s2.index)
  }
  array
}

// Defines the computation logic for each partition of RDD x (the result RDD)
override def compute(split: Partition, context: TaskContext) = {
  val currSplit = split.asInstanceOf[CartesianPartition]
```

```
  // Iterate over RDD a and RDD b to produce the cartesian product
  for {
    x <- rdd1.iterator(currSplit.s1, context)
    y <- rdd2.iterator(currSplit.s2, context)
  } yield (x, y)
}

// Defines dependencies for each partition in RDD x
override def getDependencies: Seq[Dependency[_]] = List(
  new NarrowDependency(rdd1) {
    def getParents(id: Int): Seq[Int] = List(id / numPartitionsInRdd2)
  },
  new NarrowDependency(rdd2) {
    def getParents(id: Int): Seq[Int] = List(id % numPartitionsInRdd2)
  }
)
```

Chapter Five Experiment

5.1 Initial Setup for DCQP

The usual action is represented in the following *table()*. The second column provides the *processPartition()* approach, that describes how to process the data in each partition and generate the final RDD result (also referred to as partial results)[19]. The third column defines the method *resultHandler(),* which describes how to aggregate the preliminary findings from every segment to obtain the final results[20]. Spark SQL narrows the difficulty of data analysis. Spark needs to perform an all-to-all operation to organize all the data for a reduce task, which is known as shuffling[21]. Functional programming constructs in Scala to implement Spark SQL[5, 22]. Frequently, one or more login nodes are set aside to make it easier for users to enter in via SSH and do tasks like modifying job submission scripts and source code[13, 23].

Action	finalRDD(records) => result	compute(results)
reduce(func)	(record1, record2) => result, (result, record i) => result	(result1, result 2) => result, (result, result i) => result
collect()	Array[records] => result	Array[result]
count()	count(records) => result	sum(result)
foreach(f)	f(records) => result	Array[result]
take(n)	record (i<=n) => result	Array[result]
first()	record 1 => result	Array[result]
takeSample()	selected records => result	Array[result]
takeOrdered(n, [ordering])	TopN(records) => result	TopN(results)
saveAsHadoopFile(path)	records => write(records)	null
countByKey()	(K, V) => Map(K, count(K))	(Map, Map) => Map(K, count(K))

Table 2- 5.1 SSH and do tasks like modifying job submission scripts and source code.

A task arises every time the user's driver program invokes *action().* For example, the *foreach()* action is going to carry out *sc.runJob(this, (iter: Iterator[T]) => iter.foreach(f))* when a job is given to the *DAGScheduler*[5]. The driver software will send more jobs if it has more *action()'*s. As a result, a driver program's number of jobs and *action()* actions will be identical. Because of this, a driver program is referred to in Spark as an application rather than a job. The last phase of a task produces the task's outcome[15]. For instance, there are two tasks with two sets of results in *the GroupByTest* in the first chapter. Upon submission of a task, the *DAGScheduler* utilizes a technique to determine the stages and sends the parentless stages for execution first.

The kind and quantity of jobs are also decided upon throughout this procedure. A stage runs subsequent to the execution of its parent stages[6].

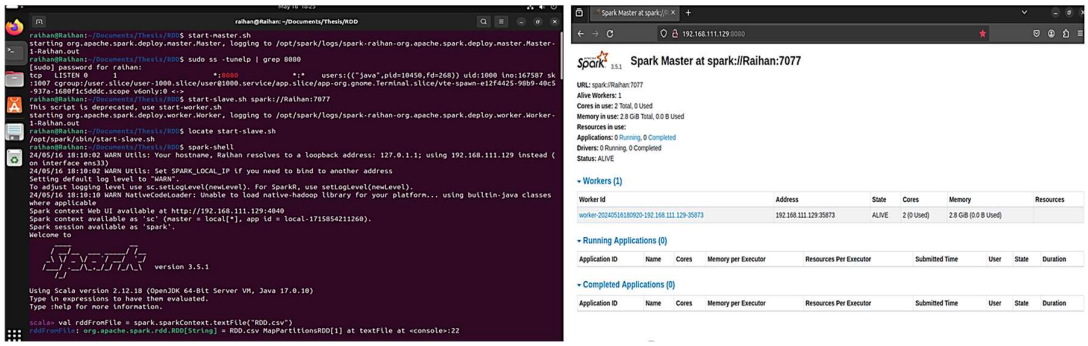


Figure 10- 5.1 Initial Job Creation

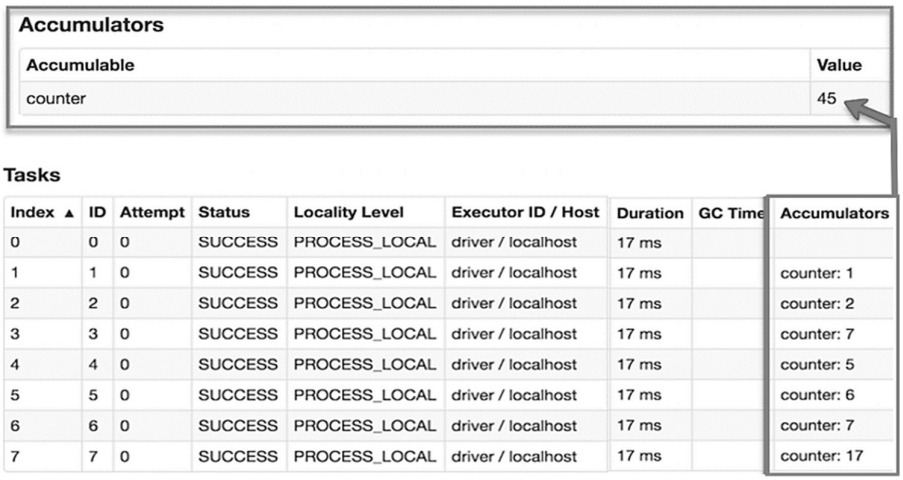


Figure 11- 5.1 Counter in Accumulator

5.2 Specific Experimental Details

Let's take a quick look at the job creation and submission code. This is a section that we will revisit in the Architecture section and talk about the processings:

- *DAGScheduler* is called by *rdd.action()*.To create a job, use *runJob(rdd, processPartition, resultHandler)*.
- *RunJob()* calls *rdd.getPartitions()* to obtain the final RDD's partition number and type. Next, *Array[Result](partitions.size)* is allocated to store the results according to the partition number.

- In the end, *runJob(rdd, partitions, allowLocal, cleanedFunc, resultHandler)* within *DAGScheduler* is used to submit the job. *ProcessPartition*'s closure-cleaned version is called *cleanedFunc.* This allows the function to be serialized and distributed among the several worker nodes.
- The *runJob()* method of *DAGScheduler calls submitJob(rdd, partitions, func, allowLocal, resultHandler)* in order to submit a job.
- After receiving another *jobId*, *submitJob()* encapsulates the procedure once again and delivers a *JobSubmitted* message to the *DAGSchedulerEventProcessActor.* This message is received by the actor, who then contacts *dagScheduler.* To manage the submitted task, use *handleJobSubmitted()*. An illustration of an event-driven programming approach is this.
- *handleJobSubmitted() submitStage(finalStage)* after first calling *finalStage = newStage()* to construct stages. The parent stages will be provided first if *finalStage* has any. In this instance, *submitWaitingStages()* submits *finalStage* in reality.

How is an RDD chain divided into stages by newStage()?

- This function calls the final instance of RDD's *getParentStages()* when a fresh stage *(new Stage(...))* is created. Assuming the final RDD, *getParentStages()* verifies the logical plan in reverse order. The RDD is included into the current phase if it's a *NarrowDependency*. When the current stage draws into account the right-side RDD, namely is the RDD that comes after the shuffle, and it finds a *ShuffleDependency* between RDDs, the stage is over. Next, using the same logic involving the left-hand RDD of the shuffle, another stage is created[15].
- Upon creation, *MapOutputTrackerMaster* will be registered as the last RDD for a *ShuffleMapStage.registerShuffle(rdd.partitions.size, shuffleDep.shuffleId)*. This is crucial because *MapOuputTrackerMaster's* data output location is required by the shuffling procedure[5].

Let's now examine how stages and tasks are sent using submitStage(stage):

- To get the *missingParentStages* of the current stage, execute *getMissingParentStages(stage). missingParentStages* will be empty if all parent stages have been completed.
- The current stage is added to *waitingStages* and these missing stages are recursively submitted if *missingParentStages* is not empty. Stages inside *waitingStages* will run once the parent stages are finished.
- In the event that *missingParentStages* is empty, the stage is ready for execution. The real tasks are then generated and submitted by using *submitMissingTasks(stage, jobId).* We will assign as many *ShuffleMapTasks* as the partition number in the final RDD if the stage is a *ShuffleMapStage.* Instead, instances of *ResultTask* are allocated in the case of *ResultStage.* A *TaskSet* is made up of the tasks in a step. *TaskScheduler* comes last. To submit the entire task set, execute *submitTasks(taskSet)*[15].
- *TaskSchedulerImpl* is the type of *taskScheduler*. In *submitTasks()*, before every single *taskSet* is sent to *schedulableBuilder*, it is enclosed in an administrative variable of type *TaskSetManager.addManagerTaskSet* with the manager. Depending on the setup, *schedulableBuilder* may be *FIFOSchedulableBuilder* or *FairSchedulableBuilder.* Notifying the backend completes *the submitTasks() process*. To do the task, use *reviveOffers(). SchedulerBackend* is the kind of backend. The application's type will be *SparkDeploySchedulerBackend* if it runs on a cluster[13].
- A child class of *CoarseGrainedSchedulerBackend* is called *SparkDeploySchedulerBackend*[13]. In reality, *reviveOffers()* notifies *DriverActor* of the *ReviveOffers* message. When *SparkDeploySchedulerBackend* begins, a *DriverActor* is launched. Upon receiving the *ReviveOffers* message, *DriverActor* will initiate the tasks by using *launchTasks(scheduler.resourceOffers(Seq(new WorkerOffer(executorId,*

executorHost(executorId), freeCores(executorId))))). a *planner.resourceOffers()* retrieves the sorted *TaskSetManager* from the Fair or FIFO scheduler and uses *TaskSchedulerImpl.resourceOffer()* to extract further task-related data. This information is stored in a *TaskDescription*. In this step the data locality information is also considered[5].

- Each job is serialized by *DriverActor's launchTasks()* method. The executor to whom the job is finally sent if the serialization size is within Akka's *akkaFrameSize* limitation is *ExecutorActor(task.executorId)!* Use the new *SerializedBuffer (serializedTask)_new* as the start task[13].

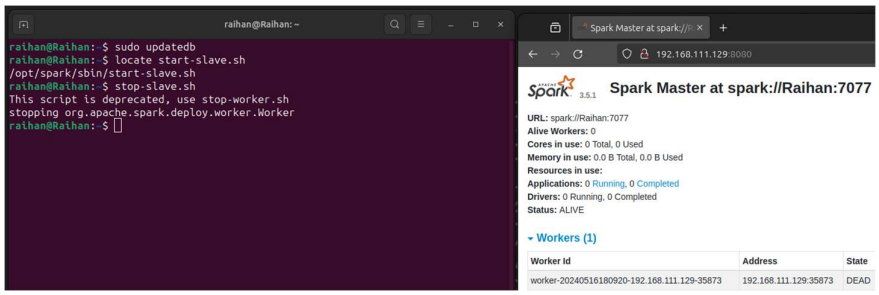


Figure 12- 5.2 Kill operation

5.3 Key Findings

In my thesis, I explored various aspects of task execution and management in Apache Spark, revealing several key findings. The driving program initiates tasks by creating a SparkContext and converting a logical plan into a physical plan using the Catalyst optimizer, which includes predicate pushdown, column pruning, and join reordering. This plan is divided into stages for parallel execution across the cluster. Spark's pipelining system chains multiple transformations into a single stage, reducing intermediate data storage and I/O operations by leveraging Resilient Distributed Datasets (RDDs) and DataFrames for efficient in-memory processing. The formal job creation process involves writing a Spark application with specified data sources, transformations, and actions, which the Spark driver packages into a job and submits to the cluster manager for task distribution to worker nodes. During the shuffling process, which redistributes data for operations like joins and aggregations, the Spark scheduler

manages intermediate data storage on local disks or memory, optimizing task execution for data locality and minimizing network latency. These findings underscore the efficiency of Apache Spark in handling large-scale distributed data processing through optimized task initiation, execution, and management. From my perspective, the transition from a logical blueprint to a physical plan is truly a masterwork. Dependencies, stages, tasks, and other abstractions are all clearly described, and the algorithms's logic is quite evident[13, 15]. Here is the RDD app implementation and result:

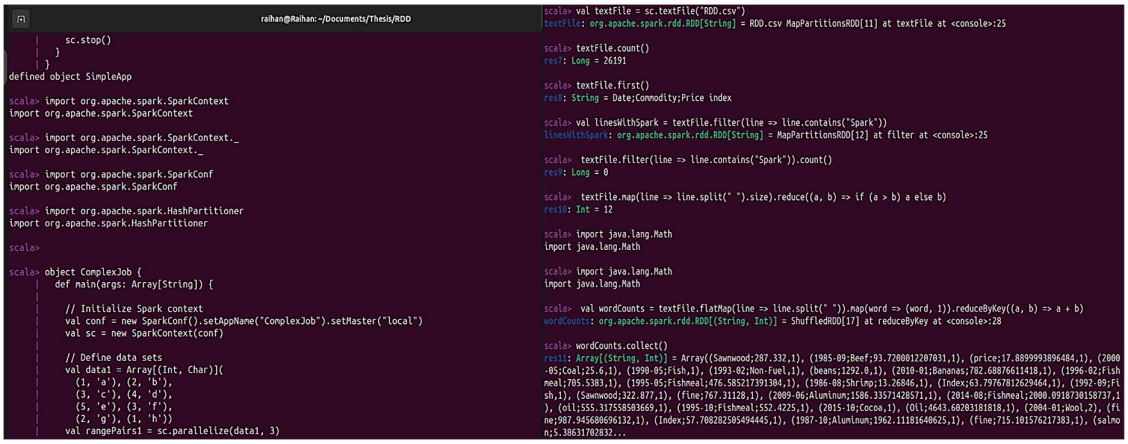
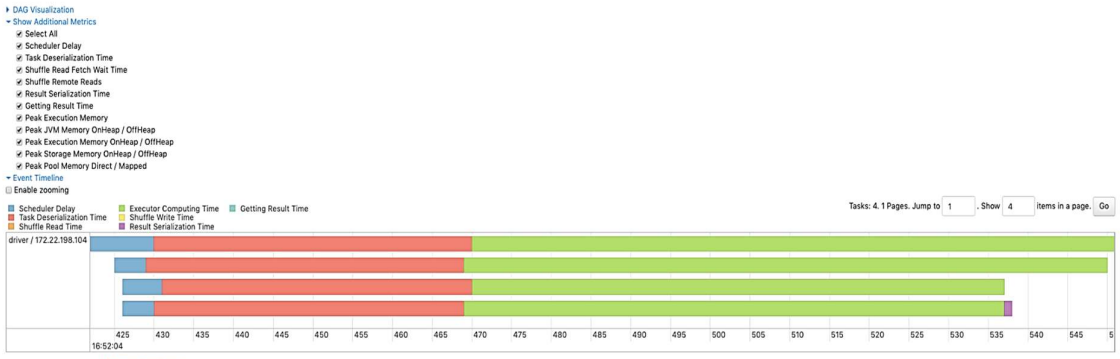


Figure 13- 5.3 RDD app implementation and result



Summary Metrics for 4 Completed Tasks

Metric	Min	25th percentile	Median	75th percentile	Max
Duration	67.0 ms	68.0 ms	81.0 ms	81.0 ms	81.0 ms
GC Time	0.0 ms	0.0 ms	0.0 ms	0.0 ms	0.0 ms
Shuffle Read Size / Records	0.0 B / 0	0.0 B / 0	46 B / 1	138 B / 3	138 B / 3
Scheduler Delay	4.0 ms	4.0 ms	5.0 ms	8.0 ms	8.0 ms
Task Deserialization Time	39.0 ms	39.0 ms	40.0 ms	40.0 ms	40.0 ms
Shuffle Read Fetch Wait Time	0.0 ms	0.0 ms	0.0 ms	0.0 ms	0.0 ms
Shuffle Remote Reads	0.0 B	0.0 B	0.0 B	0.0 B	0.0 B
Result Serialization Time	0.0 ms	0.0 ms	0.0 ms	1.0 ms	1.0 ms
Getting Result Time	0.0 ms	0.0 ms	0.0 ms	0.0 ms	0.0 ms
Peak Execution Memory	0.0 B	0.0 B	752 B	928 B	928 B

Figure 14- 5.3 Summary Matrix

Chapter Six Conclusion

1.1 Limitations of the Current Work (DCQP)

Despite the advanced capabilities and robust design of the Distributed Confidential Query Protocol (DCQP), several challenges and shortcomings need to be addressed:

- *Performance Overhead:* The encryption and decryption processes, along with secure multi-party computation, introduce significant computational overhead. This can slow down query processing times, particularly for large-scale datasets and complex queries, potentially negating some of the performance benefits of distributed computing with Spark.
- *Scalability Issues:* While DCQP leverages Spark's distributed architecture, the cryptographic techniques used can limit scalability. As the dataset size grows, the computational and memory requirements for encryption, decryption, and secure computation increase disproportionately, which can lead to scalability bottlenecks.
- *Complexity of Implementation:* Integrating DCQP with existing Spark workflows can be complex. Users need to have a deep understanding of cryptographic principles and the specific requirements of the DCQP API, which can be a barrier to widespread adoption.
- *Limited Query Support:* Not all types of queries can be efficiently executed on encrypted data. Some operations may require data to be decrypted or partially decrypted, undermining the confidentiality guarantees. This limits the range of analytics that can be performed securely.
- *Interoperability Concerns:* DCQP may face interoperability issues with other data processing and analytics tools. Ensuring seamless integration across different platforms and maintaining the confidentiality of data throughout the entire processing pipeline remains a challenge.

- *Data Leakage Risks:* Although DCQP aims to protect data confidentiality, there are still risks of data leakage through side-channel attacks, inference attacks, or vulnerabilities in the implementation of cryptographic algorithms.

6.2 Future Work Directions

To address these shortcomings and enhance the effectiveness and adoption of DCQP, several areas of future work are proposed:

- *Performance Optimization:* Research and development should focus on optimizing cryptographic algorithms and secure computation techniques to reduce computational overhead. This includes exploring lightweight encryption methods and optimizing parallel processing capabilities within Spark.
- *Enhanced Scalability:* Improving the scalability of DCQP is crucial. This could involve developing more efficient algorithms for handling large-scale encrypted data and leveraging advanced distributed computing techniques to better manage resources.
- *Simplified Integration:* Efforts should be made to simplify the integration of DCQP with existing Spark workflows. Providing comprehensive documentation, user-friendly tools, and automated configuration options can help lower the barrier to entry for users.
- *Broader Query Support:* Extending the range of queries that can be securely executed on encrypted data is important. This could involve developing new cryptographic protocols that support a wider variety of operations without compromising confidentiality.
- *Interoperability Enhancements:* Ensuring that DCQP can interoperate seamlessly with other data processing tools and platforms is essential. This may require the development of standard interfaces and protocols for secure data exchange across different systems.
- *Strengthening Security:* Addressing potential vulnerabilities and enhancing the security of the DCQP implementation is critical. This includes conducting thorough security audits,

developing defenses against side-channel and inference attacks, and continuously updating cryptographic techniques.

- *User Education and Support:* Providing training and support resources for users can help them better understand and effectively use DCQP. This includes offering workshops, tutorials, and dedicated support channels to assist users in implementing and maintaining the protocol.

By focusing on these areas, the DCQP can be improved to provide more efficient, scalable, and user-friendly solutions for secure distributed data processing, ultimately broadening its adoption and impact across various industries.

6.3 Summary of Key Findings

The Distributed Confidential Query Protocol (DCQP) is a new milestone in providing data protection in distributed computing settings in an even more important way, wherever Apache Spark is being generally used. Through the use of Trusted Execution Environments (TEEs) DCQP proposes a solid model of data keep within the framework of distributed processing pipeline which constitutes a basis for re-prospering data confidentiality and thus privacy in Spark context. DCQP deglamorizes the cyber challenges in today's world by means of employing morphologically flexible encryption techniques, strict access control, together with advancement in accountability mechanisms, that significantly raises the baseline of hackers in their efforts to gain access and navigate through systems. Implementation of the life checks private plug-in is done within Spark cluster architecture, doing this allows processing of private queries across multiple big data nodes easily.

The purpose of this paper is to provide a thorough explanation of the elements that influence the development and implementation of DCQP strategy, highlighting its significance in Spark

operations. To add, conducting a lot of experiments showed that the information security improved a lot under this model besides performance and scalability in cloud environments remained excellent. Lastly, DCQP forms the main pillar which serves as a solid data protection mechanism within the distributed computing environments of the organizations that seek to compensate their security measures. Besides increasing data security by implementing it with Apache Spark, the integrity and privacy of various sensitive info are also guaranteed and kept private within the appropriate use environments.

References

[1] Y. Chen, Q. Zheng, Z. Yan, and D. Liu, "QShield: Protecting Outsourced Cloud Data Queries With Multi-User Access Control Based on SGX," *IEEE Transactions on Parallel and Distributed Systems,* vol. 32, no. 2, pp. 485-499, 2021, doi: 10.1109/tpds.2020.3024880.

[2] V. S. Khandekar, Shrinath, P., "Ensemble Model for Multiclass Imbalanced Data Using Cluster Computing of Spark," *IIETA,* pp. 161-167, 2023.

[3] GeeksforGeeks, "What is a Distributed System?," in *GeeksforGeeks* vol. 2024, ed. Online: https://www.geeksforgeeks.org/what-is-a-distributed-system/: GeeksforGeeks, 2024.

[4] E. Khashan, A. Eldesouky, and S. Elghamrawy, "An adaptive spark-based framework for querying large-scale NoSQL and relational databases," *PLOS ONE,* vol. 16, no. 8, p. e0255562, 2021, doi: 10.1371/journal.pone.0255562.

[5] A. Spark, "RDD Programming Guide," in *Apache Spark* ed. Online: https://spark.apache.org/docs/latest/rdd-programming-guide.html: Apache Spark 2024.

[6] Amazon, "What is Spark," in *Amazon*, ed. Online: https://aws.amazon.com/what-is/apache-spark/: Amazon, 2024.

[7] Toptal, "Spark," in *Toptal*, Toptal, Ed., ed. Online: https://www.toptal.com/spark/introduction-to-apache-spark: Toptal, 2016.

[8] D. Chang, L. Li, Y. Chang, and Z. Qiao, "Cloud Computing Storage Backup and Recovery Strategy Based on Secure IoT and Spark," *Mobile Information Systems,* vol. 2021, pp. 1-13, 2021, doi: 10.1155/2021/9505249.

[9] A. Sabih Shaker, "A Survey of Scaling Distributed System Via Machine Learning and An Insight on Hadoop and Spark," *IOP Conference Series: Materials Science and Engineering,* vol. 928, no. 3, p. 032008, 2020, doi: 10.1088/1757-899x/928/3/032008.

[10] A. A. Hagar and B. W. Gawali, "Apache Spark and Deep Learning Models for High-Performance Network Intrusion Detection Using CSE-CIC-IDS2018," *Computational Intelligence and Neuroscience,* vol. 2022, pp. 1-11, 2022, doi: 10.1155/2022/3131153.

[11] J. Han, Y. Seo, S. Lee, S. Kim, and Y. Son, "Design and Implementation of Enabling SQL–Query Processing for Ethereum-Based Blockchain Systems," *Electronics,* vol. 12, no. 20, p. 4317, 2023, doi: 10.3390/electronics12204317.

[12] M. Zaharia *et al.*, "Apache Spark," *Communications of the ACM,* vol. 59, no. 11, pp. 56-65, 2016, doi: 10.1145/2934664.

[13] J. Laskowsk, *Mastering Apache Spark 2.0*, Online: https://mallikarjuna_g.gitbooks.io/spark/content/: Gitbooks, 2020-2024.

[14] M. M. Khan, M. A. U. Alam, A. K. Nath, and W. Yu, "Exploration of memory hybridization for RDD caching in Spark," 2019 2019: ACM, doi: 10.1145/3315573.3329988. [Online]. Available: https://dx.doi.org/10.1145/3315573.3329988

[15] H. J. Lijie Xu, Hao Ren, Bhuridech Sudsee, "Spark Internals," *Spark Internals*.

[16] X. Ji, M. Zhao, M. Zhai, and Q. Wu, "Query Execution Optimization in Spark SQL," *Scientific Programming,* vol. 2020, pp. 1-12, 2020, doi: 10.1155/2020/6364752.

[17] H. Sun, R. He, Y. Zhang, R. Wang, W. H. Ip, and K. L. Yung, "eTPM: A Trusted Cloud Platform Enclave TPM Scheme Based on Intel SGX Technology," *Sensors,* vol. 18, no. 11, p. 3807, 2018, doi: 10.3390/s18113807.

[18] S. Woo, J. Song, and S. Park, "A Distributed Oracle Using Intel SGX for Blockchain-Based IoT Applications," *Sensors,* vol. 20, no. 9, p. 2725, 2020, doi: 10.3390/s20092725.

[19] G. Lin and Y. Zhang, "Sparks of Artificial General Recommender (AGR): Experiments with ChatGPT," *Algorithms,* vol. 16, no. 9, p. 432, 2023, doi: 10.3390/a16090432.

[20] R. C. Yong Zhao, "Research on Runtime Query Optimization Technology of Spark SQL " *International Journal of Computer Theory and Engineering,* vol. 14, 2022.

[21] M. Chawla and V. Baniwal, "Optimization in the catalyst optimizer of Spark SQL," *TURKISH JOURNAL OF ELECTRICAL ENGINEERING & COMPUTER SCIENCES,* vol. 26, no. 5, pp. 2489-2499, 2018, doi: 10.3906/elk-1707-6.

[22] M. Armbrust *et al.*, "Spark SQL," in *ACM*, 2015 2015: ACM, doi: 10.1145/2723372.2742797. [Online]. Available: https://dx.doi.org/10.1145/2723372.2742797

[23] J. J. Alnasir, "Fifteen quick tips for success with HPC, i.e., responsibly BASHing that Linux cluster," *PLOS Computational Biology,* pp. 1-9, 2021.

Acknowledgement

At first, I'll reward me to almighty Allah maximum merciful more useful thanks loads to having me on this global.

Secondly, I would like to specific my appreciation to Dr. Chen Yaxing, my supervisor, from the lowest of my heart for all of his assist and assist throughout the writing of my thesis. His information, tolerance, and beneficial criticism were useful in figuring out the direction and quality of this work.

I also need to specific my gratitude to all of the academics for his or her thoughtful evaluations and tips, which sizeable okay for progressed the thesis's substance.

I clearly thank my own family for his or her everlasting guide, love, and expertise. Their unwavering help has been my pillar of aid at some point of this academic enterprise.

My buddies and coworkers have helped me construct this thesis thru their guide, guidance, and insightful conversations. I am appreciative of their encouragement.

An extra thanks to Northwestern Polytechnical University for presenting the gear and space needed to try this observe.

Lastly, I simply want to say thanks to all and sundry who took the time to participate on this examine and shared their ideas with me. The help and participation of anybody indexed above were important to the final touch of this thesis. Grateful having the ones humans around me. Thanks lots.